

AMSC460: Homework 4

JAMES ZHANG^{*}

September 27, 2024

0. Review questions

- (a) What is the overall cost in flops of applying a forward or backward solve?
- (b) What is the overall cost in flops of decomposing a matrix using LU?
- (c) During the course of Gaussian elimination without pivoting a zero pivot has been encountered. Is the matrix singular?
- (d) State three disadvantages of computing the inverse of a matrix to solve a linear system rather than using the LU decomposition approach.

Solution.

- (a) The overall cost in flops for forward or backward solving is $\mathcal{O}(n^2)$.
- (b) The overall cost in flops for decomposing a matrix using LU is $\mathcal{O}(n^3)$.
- (c) No. If we encounter a zero pivot during the course of Gaussian Elimination, this does not always mean that the matrix is singular — we can interchange the rows. See **Example 5.7** in the textbook for a counterexample.
- (d) Three advantages to using a LU Decomposition approach over a matrix inverse approach are
 1. Finding the inverse of a matrix is an inefficient, wasteful use of storage.
 2. Finding the inverse of a matrix — while also an $\mathcal{O}(n^3)$ algorithm — is more computationally expensive than finding the LU Decomposition, though by less than an order of magnitude.
 3. Finding the inverse of a matrix is more prone to larger roundoff errors than the LU Decomposition. Finding the inverse also has no advantages to outweigh the disadvantages.

□

^{*}Email: jzhang72@terpmail.umd.edu

1. Consider the problem

$$\begin{aligned}x_1 - x_2 + 3x_3 &= 2, \\x_1 + x_2 &= 4, \\3x_1 - 2x_2 + x_3 &= 1.\end{aligned}$$

Carry out Gaussian elimination in its simplest form for this problem. What is the resulting upper triangular matrix?

Proceed to find the solution by backward substitution.

Solution. The above system yields the following augmented matrix, which we will perform Gaussian Elimination on in order to obtain an upper triangular matrix.

$$\begin{aligned}\left[\begin{array}{ccc|c}1 & -1 & 3 & 2 \\1 & 1 & 0 & 4 \\3 & -2 & 1 & 1\end{array}\right] &\xrightarrow{R_1 - R_2, \text{ \& } R_1 - \frac{1}{3}R_3} \left[\begin{array}{ccc|c}1 & -1 & 3 & 2 \\0 & -2 & 3 & -2 \\0 & -\frac{1}{3} & \frac{8}{3} & \frac{5}{3}\end{array}\right] \\&\xrightarrow{R_2 - 6R_3} \left[\begin{array}{ccc|c}1 & -1 & 3 & 2 \\0 & -2 & 3 & -2 \\0 & 0 & -13 & -12\end{array}\right]\end{aligned}$$

and this is our resulting upper triangular matrix. Now applying backwards substitution, we get immediately that

$$-13x_3 = -12 \implies x_3 = \frac{12}{13}$$

Now plugging this value in for the next equation, we solve

$$-2x_2 + 3\left(\frac{12}{13}\right) = -2 \implies -2x_2 = -\frac{62}{13} \implies x_2 = \frac{31}{13}$$

Finally, solving for the last unknown x_1 ,

$$x_1 - \frac{31}{13} + 3\left(\frac{12}{13}\right) = 2 \implies x_1 = 2 + \frac{31}{13} - \frac{36}{13} \implies x_1 = \frac{21}{13}$$

Therefore, using Gaussian Elimination and Backwards Substitution, we arrive at our solution

$$\vec{\mathbf{x}} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} = \begin{bmatrix} \frac{21}{13} \\ \frac{31}{13} \\ \frac{12}{13} \end{bmatrix}$$

□

3. Let A and T be two nonsingular, $n \times n$ real matrices. Furthermore, suppose we are given two matrices L and U such that L is unit lower triangular, U is upper triangular, and

$$TA = LU.$$

Write an algorithm that will solve the problem

$$Ax = b$$

for any given vector b in $\mathcal{O}(n^2)$ complexity. First explain briefly yet clearly why your algorithm requires only $\mathcal{O}(n^2)$ flops (you may assume without proof that solving an upper triangular or a lower triangular system requires only $\mathcal{O}(n^2)$ flops). Then specify your algorithm in detail (including the details for lower and upper triangular systems) using pseudocode or a MATLAB script.

Proof. Note that in the following steps, we are not computing the inverse of T in MATLAB, but rather we are doing this to develop a new algorithm and rewrite our givens. Let T^{-1} be the matrix inverse of T and observe that

$$TA = LU \implies T^{-1}TA = T^{-1}LU \implies A = T^{-1}LU$$

Substitute this into the equation $Ax = b$, and we get that

$$T^{-1}LUx = b$$

Now left multiply both sides by T , and we are left with

$$TT^{-1}LUx = Tb \implies LUx = Tb$$

where Tb is a new vector. Let $y = Ux$, which yields

$$Ly = Tb$$

and we can then solve for y using forward substitution. Once we have y , we can solve

$$Ux = y$$

for x using backward substitution, and we will have obtained our answer.

Time Complexity: Note that the matrix multiplication Tb , forward substitution, and backward substitution are all $\mathcal{O}(n^2)$, and so our entire algorithm's time complexity is $\mathcal{O}(n^2)$.

```

1 % Given A, T, L, U, and b.
2 Tb = T * b;
3 % Forward Substitution.
4 for k = 1:n
5     y(k) = (Tb(k) - L(k, 1:k-1) * y(1:k-1)) / L(k, k);
6 end
7 % Backwards Substitution.
8 x(n) = y(n) / U(n,n);
9 for k = n-1:-1:1
10    x(k) = (y(k) - U(k,k+1:n) * x(k+1:n)) / U(k,k);
11 end

```

□